

Python Book Analysis

Oscar Stomberg

August 13, 2020

Contents

1	The Project	2
1.1	The Assignment	2
1.2	The Idea	2
2	First Programs	3
2.1	Random Word Generator	3
2.2	Word Separator	3
2.3	Word Frequency Comparison	4
2.4	Letter Separator	6
2.5	Random Word Generator V2	6
3	Combination Analysis	7
3.1	Letter Pairs	8
3.2	Using Anaconda	9
3.3	Indexed Letter Pairs	10
3.4	Letter Triplets	10
3.5	Random Word Generator V3	10
4	More Simple Programs	11
4.1	Favorite Word Picker	11
4.2	Real Word Checker	11
5	Multi Threaded Processing	12
5.1	Why Multi-Thread?	12
5.2	Using Multiprocessing	12
5.3	The Results	13
5.4	Why So Much Improvement?	14
5.5	Multi-Threading the Word Generator	14
6	Reflections	16

1 The Project

1.1 The Assignment

The goal of this project so far has been relatively simple: To learn and become more comfortable with a new programming language. However the tasks associated with the project have varied greatly. It started with a summer project for the AP Computer Science class I will be attending next year, which was to:

1. Learn a new programming language and create an original project using it.
2. Write a reflection containing:
 - (a) What the program does.
 - (b) Why I like my program.
 - (c) What resources I used and how I gained inspiration.
 - (d) What surprised me, what was most difficult during the process of writing it.
 - (e) What I would like to learn in class next year.
3. Demonstrate the project to class after summer is over and school starts.

1.2 The Idea

This summer assignment really inspired me to do something to learn more about how to code, so I started thinking about what I would use to create my project, and what I would do for my project. I had been working on various projects with my Arduino boards and sensors using the Arduino programming language. I originally planned on completing the summer project using these things, however, I had started to run into issues with the hardware and was not making much progress with it. Another reason I ended up not using Arduino for my project is because it would end up being more centered on getting the hardware working than on creating interesting software projects.

For some context, a year or two back I had been spending a lot of time coding with Python. However, I had been focusing on creating games using tkinter, which was arguably not the best use of my time given the plethora of dedicated game engines out there. Although I learned a lot about basic syntax and coding with Python, I still have a lot to learn about using python for its main purposes, which are data and text analysis. After a while, I ended up moving away from Python and spending more time creating games using the Unity game engine, and have been dabbling in a few other things since then such as web design, Processing, and more recently, Arduino.

I received my inspiration for what I would do when I launched the PyCharm IDE for Python one day, and opened one of my old programs, a random word generator. This program would print out a list of random, but pronounceable

gibberish in the terminal when run. I immediately started cleaning up and improving the code, and made it output into a text file instead of into the terminal. This gave me all sorts of ideas for new programs to create, and so I continued from there.

2 First Programs

2.1 Random Word Generator

The first program in this project, the Random Word Generator (Word Generator.py) was initially created a long time ago. However, I did have to spend a significant amount of time:

- Making the code more clean and concise.
- Protecting the inputs from errors.
- Making the program output to a text file instead of printing to the terminal.

This program works by creating a chosen number of random words, each with a random length between a minimum and maximum you provide. Each word is created by alternating between appending either a random consonant or a random vowel to the word (starting on either a vowel or consonant) until the randomly chosen length has been reached. It then adds the word to a list, and repeats this process until the desired number of words have been generated. It then opens or creates a text file (output.txt) and writes this list of words as well as information about the number of words and the limits to the random lengths of the words. Here is some sample output:

```
Jekuzedecobu
Kuqobeq
Ikidiqabonakenedo
Vipale
Efoliyihesakezane
Imiwifapekod
Yefudutize
Ugafubirojijicogu
Jodewidu
Deha
```

2.2 Word Separator

I made this program, the Word Separator (Word Separator.py), after suggestion from my dad. I was showing him my finished random word generator, and he suggested I have the distribution of the random lengths of words reflect the

distribution of word lengths in the English language. So, I decided to start by analyzing various pieces of literature with a different Python program.

I started by downloading various lengthy and well known books including *Moby Dick*, *Ulysses*, *20000 Leagues Under The Sea*, and a few others. These books were all freely available to download online as text files due to their copyrights having expired. I then created a program that would:

1. Import a book as a text file from the project folder. The user selects the book.
2. Convert everything to lowercase and remove all numbers and punctuation.
3. Separate the text into a simple list of words.
4. Iterate through the list of words and create a two-dimensional list of all the unique words in the book, where each item contains a word, how many times it appears in the text, and how many times it appears per 10000 words.
5. Write this list of unique words and their data to a separate output text file.

I could then use the word frequency analyses provided by this program in other programs, for example to compare the analyses from different texts.

2.3 Word Frequency Comparison

The Word Frequency Comparison program (`Frequency Analyzer.py`) is a program I created that imports the Word Separator Outputs for two books of the user's choice and compares the frequencies of words that appear in both texts, as well as providing which words are unique to each book. It works like this:

1. Import the data for two books (chosen by the user) from the output of the Word Separator as separate two-dimensional lists.
2. For each word in one list, check if it is in the other list. If it is, add it to a list of words that appear in both books, along with the ratios between the frequency per 10000 of the word in each book.
3. For each word in each list, check if the word is in the other list. If not, add it and the number of time it occurs in it's text to a list of unique words.
4. Create another output file and write the list of shared words and the lists of unique words.

This output can be difficult to read, but it is interesting to see how the proportions of shared words reflect the overlaps and differences in the content of the books. Here I have made tables showing some of the output from comparing *Moby Dick* to *Ulysses*. Ratio 1 is equal to the occurrences in *Moby Dick* divided by the occurrences in *Ulysses*, and vice versa for Ratio 2.

Table 1: More Common In Moby Dick

Word	Ratio 1	Ratio 2	Total
Whale	506.143	0.002	1246
Whales	334.027	0.003	272
Sperm	301.973	0.003	246
Mast	162.703	0.006	133
Chapter	125.42	0.008	311
Jonah	106.0	0.009	87
Harpoon	92.432	0.011	76
Pip	92.432	0.011	76
Peculiar	69.027	0.014	57
Lance	55.459	0.018	46

Table 2: More Common In Ulysses

Word	Ratio 1	Ratio 2	Total
Miss	0.009	109.152	136
Joe	0.009	110.783	138
Irish	0.011	94.609	118
Ireland	0.014	72.783	91
Bob	0.019	53.37	67
Buck	0.021	47.418	118
Pat	0.023	42.848	54
Martin	0.023	43.736	109
Trousers	0.024	41.239	52
J	0.024	42.302	211

Table 3: Equally Common

Word	Ratio 1	Ratio 2	Total
Nervous	1.002	0.998	20
Never	1.002	0.998	458
Soil	1.002	0.998	20
Sum	1.002	0.998	20
Twisted	1.002	0.998	20
Cause	0.999	1.001	49
Hearing	0.997	1.003	29
Spoken	0.997	1.003	29
Of	0.995	1.005	14 756
Die	0.994	1.006	67

2.4 Letter Separator

The Letter Separator (Letter Separator.py) is very similar to the word separator, borrowing much of the code. It iterates through the alphabet and counts how many of each letter there are in the book. However, rather than separating the words into a list, it removes all non-alpha characters from the text and uses the string as it would use a list. Much like the Word Separator, it then writes this data to a text file, along with the occurrences per 100 for each letter.

This program is used in the second version of the Random Word Generator.

2.5 Random Word Generator V2

The Random Word Generator V2 (Word Generator V2.py) is built on the original Random Word Generator using previously created programs such as the Word Separator and Letter Separator.

This version, however, attempts much more to emulate word length and letter distribution found in real literature. To get this information, it references the outputs from the Word Separator and Letter Separator. It works in these steps:

1. Choose a book and import the analyses.
2. Determine the frequency distribution of letters in the book.
3. Determine the frequency distribution of different word lengths.
4. Determine the chance for a word to start with a vowel for each given word length.
5. Receive user inputs for number of words, minimum letters, and maximum letter, like in the original program. Maximum is limited by the longest word in the book.
6. Determine the random word length based on word length distributions, and based on that length get whether it starts on a vowel or consonant.
7. Start creating a word, alternating vowels and consonants. For each letter, choose a random letter based on the letter distributions.
8. Create the chosen number of words, add them to a list, and output it to a text file.

Here is the backbone of how the program chooses a random letter based on a list of probabilities, which is used in many later programs:

```
1 def get_letter(vowel, probs): # Get a random letter reflecting the letter
    probabilities
2     if vowel: # Limit the string of characters to vowels or consonants
3         string = vowels
4     else:
5         string = consonants
6     total = sum(probs) # Get the total from the list of probabilities
```

```

7     rand = random.random() * total # Get random value
8     added = 0
9     for por in probs: # Add from list of probabilities until at or above
                        random value
10        added += por
11        if added >= rand: # Get the corresponding character
12            return string[probs.index(por)]

```

As you can see in this snippet of code, probabilities are calculated by simply taking the counts of each item and adding them together to get the total. For example, if the count for **a** is 78205 and the total count is 955836 letters, the chance for **a** to appear is $\frac{78205}{955836}$. Here are some words from this program's output, using the text form Moby Dick as an input:

Temasecos
 Tita
 Degiwepi
 Qeresute
 Rotomo
 Onepulahad
 Dite
 Terosen
 Ces
 Tinafis

This program successfully makes the set of words seem more realistic, with many more of certain letters than other as we see in English, as well as more medium and short length words than extremely long words or extremely short words.

However, it could still be greatly improved through letter pair analysis of words in a book. This would have the program pick a letter based on the previous letter, rather than based on overall letter probabilities. For example, it would first pick a starting letter based on a list of letters' probabilities of being at the start of a word. For this example, let's say it picks **A** to start the word. Then, repeating for the predetermined length of the word, it would look at the previous letter and pick a new letter using a list of probabilities for letters given a preceding letter. To continue our example, it would see that **a** is the previous letter and check the probability list for **a**. It might then pick the letter **p** and then repeat the process until the word has reached the randomly chosen length. Doing this would substitute for simply alternating vowels and consonants and lead to more sounds and syllables we hear in real English words.

3 Combination Analysis

In this section, I will document my process in learning how to use multi-dimensional arrays and data frames to analyze different combinations of letters and words in books. I will also apply this information to a word prediction algorithm and my Random Word Generator.

3.1 Letter Pairs

I started off with attempting to apply basic word pair analysis to my Random Word Generator as explained at the end of Section 2.5. Rather than using the overall probability of a letter, this program (Word Generator V3.py) uses the conditional probability of a letter given the letter that precedes it. Here is the step-by-step process for how it works:

1. Get words list from book.
2. For each word, add each letter pair to a matrix with the probabilities of letters.
3. Add the first letter of each word to an array with the probabilities of letters being the first letter in a word.
4. Proceed like the Word Generator V2, but find the probability for each letter based on the previous letter.

This approach actually does improve how it mimics real sounds and syllables in the English Language. However, it does have drawbacks which I will explain. Here is some sample output:

Pounesuron
Eswall
Ramasha
Darstle
Fobla
Izav
Wanu
Buspli
Rtliipi
Blyintrab

As you can see, most of the words are interesting and pronounceable. However, due to the nature of the algorithm, there are some instances of non-pronounceable words.

Take the word `Rtliipi` for example. This is clearly not pronounceable, but why? When it was created it started by choosing a random letter based on how often it appears at the start of words, and came up with `R`. Then, based on that letter, it checked for letters that commonly follow `R` and added `t` to the word. Any human would know that `Rt` can never appear at the front of a word, only the middle or end. However, the computer does not have enough context to know this, and so mistakes such as this occur frequently. Here are three possible solutions:

1. Add a second dimension to the matrix of starting letter probabilities, to store starting letter pairs rather than just starting letters. This would

have the program check for what letter pairs occur at the start of a word, not just what letter pairs occur, so pairs like `Rt` would not appear at the start of a word.

- Pro: Relatively simple to implement, probably effective.
 - Con: Can diminish variation at end of longer words.
2. Add a third dimension to the matrix of letter probabilities. This would make it analyze letter triplets rather than pairs to give the program more context so it can avoid things like having `ppp` in the middle of a word.
 - Pro: Is more effective at generating interesting words.
 - Con: Difficult to implement, may require Anaconda Libraries.
 3. Create separate program that lets the user sort through the list of words and separate out the non-pronounceable words.
 - Pro: Very simple and easy to do.
 - Con: Imperfect, requires human input.

The best method, however, is probably a thoughtful combination of all of these ideas.

3.2 Using Anaconda

In order to execute the ideas detailed in Section 3.1, however, it will become necessary to add many layers and dimensions to the arrays which hold all the data. This is doable with basic lists in Python, however the process for generating such an array is inefficient and is not very concise. Here is what it looked like before:

```
1 alpha = 'abcdefghijklmnopqrstuvwxyz'
2 d = [0] * len(alpha)
3 dictionary = [d for a in len(alpha)]
```

Luckily, we are using Python, with its vast array of libraries for anything you might want to do. Anaconda is an application with a collection of libraries dedicated to data analysis. So, it was simply a matter of installing Anaconda and integrating it into my IDE. Afterwards, I was able to easily import the NumPy library into my program and use it to create a multidimensional array in a single line of code:

```
1 dictionary = numpy.zeros((26, 26))
```

As you can see, this approach is not only more concise, but also much more easily scalable, needing only an extra argument to add another dimension, whereas the previous method requires an extra line for each new dimension. This also has the added benefit of all the NumPy functions I can run on the array. Now that I have Anaconda set up, I can continue with the project.

3.3 Indexed Letter Pairs

The first step was to look at not just the frequencies of letters at the start of words, but also the frequencies of different letter pairs at the start of words. This would fix the problem of words starting with an unpronounceable syllable. Previously, I had been using a simple one dimensional array for first letter analysis, with one cell for each letter's frequency.

To look at first pairs in words, I had to add another dimension. Using NumPy, I created an array with 26x27 cells. This allowed me to store the frequencies of each letter pair at the start of a word. The extra values in the second dimension serve to store the frequency of each single letter at the start of a word.

3.4 Letter Triplets

The other issue that needed fixing was that by only looking at letter pairs, the program did not have the context it needed to avoid, for example, putting three of the same letter in a row. However, this can be easily solved by using letter triplets.

I simply added a third dimension to my letter pair array, making it a 26x26x26 array. With this array, I simply ran the same program as before with each letter determined by the previous two letters. The first dimensions stores the letter two letters before, the second dimension stores the previous letter, and the third dimension stores probabilities for the next letter based on the previous two letters.

3.5 Random Word Generator V3

Combining the structure of the Random Word Generator V2 with letter triplet analysis and first letter and letter pair analysis at the beginning of each word, I came to the Random Word Generator V3 (Word Generator V3.py). It all worked as expected and resulted in a much cleaner and more interesting output of random words. Here is some sample output:

```
Riddilect
Harly
Cro
Spinshed
Blyne
Hobbalph
Adompti
Culatch
Squed
Bolessedgerl
```

In these words, we see many more recognizable combinations than we did in V1, V2, or when the program only looked at pairs of letters. For example,

in this small selection of words we see `-ect`, `-shed`, `-yne`, `-alph`, `-atch`, and `Squ-`, all combinations of letters we see in real words. This happens because this version of the program stores much more data about what letters tend to go together in real words.

It remains imperfect, however it is much better than it was before for the purpose of randomly generating words that sound like English words. There remain issues, mostly with the input to the random word generator. For example, it may interpret chapter numbers such as `XX`, leading the program to create words with a double `x`. However, there remain a few more programs we can create using the output of the Random Word Generators. For one, I can create a program as detailed in Section 3.1 which allows the user to select favorite words from the output. Another program could look through the words to find how many real words are generated.

4 More Simple Programs

4.1 Favorite Word Picker

This program (`Word Picker.py`) is designed to let the user more interactively view the random words, while separating out which words are pronounceable and which ones are not. It simply loads all the words from the output, and prints the out one at a time. Then, it asks the user whether to keep it, or not. If the user chooses to keep it, it gets put into a separate list which is then put into a separate text file. This is the final program necessary to get the most out of the random word generator.

4.2 Real Word Checker

The Real Word Checker (`Word Checker.py`) is a simple and concise program that takes the output from the Random Word Generator and compares it to the list of words in a book to check which randomly generated words are actually in the book.

Here are some of the real words it found that were randomly generated:

25667 real words found in 1000000 random words:

Spos
Rear
Tail
Hval
Sable
Shed
Mend
Under
Lac
Capes

...

As we can see, most of the random words that turn out to be real are very short, rarely more than 5 letters long. However, we can also see based on the numbers from the top that roughly 2.5% or one in 40 of all randomly generated words are actually in the original text. Because this is such a high ratio, I may modify the program to remove all real words from the list of randomly generated words or implement this program in the Random Word Generator to reject all previously existing words.

5 Multi Threaded Processing

5.1 Why Multi-Thread?

At this point, I had created quite a few compelling and interesting programs. However, some of them took quite a while to run, particularly the Word Separator and Random Word Generator V3. For example, with a simple timing script I could see that the Word Separator takes 65984 milliseconds to run. This is over a minute, which is quite a while for a simple program such as this.

When I looked at my system's resource utilization while running the program, I quickly noticed the problem: It was only using 25% of my CPU. Ideally, it would be using 100%. However, my program only uses one thread, limiting my program to only one of my processor's 4 cores.

5.2 Using Multiprocessing

Thankfully, Python has the perfect solution to my problem: The multiprocessing library. This allows my python program to quickly and easily create new threads that can run in parallel on separate cores in my processor.

This also comes with certain challenges. This first one is relatively simple, and that is that I need to separate the data into equal chunks for each thread to analyze. Here is the snippet of code I came up with to do this:

```
1 tcount = cpu_count() # Sets the number of threads to the CPU's core count
2 chunk = math.floor(len(words) / tcount) # Get the length of a chunk
3 chunks = [] # Create chunks array
4 for t in range(tcount - 1): # Add chunks of data to array
5     chunks.append(words[t * chunk:(t + 1) * chunk])
6 # Add the rest of the data to the last chunk, in case the data cannot be
   perfectly divided by the thread count
7 chunks.append(words[(tcount - 1) * chunk:len(words)])
```

Another issue is that because the different threads run in separate cores, they cannot share memory. This means that each individual thread will have to write its output to its own text file on the disk, after which the main thread will read the outputs in each thread's file. Once it has these outputs, it also has to combine them in a way that eliminates redundant results, such as four different counts for the word `the` from the four threads.

5.3 The Results

With those problems solved, I wrote the code necessary to implement this multi-Threading. Using my timing script, I found that this divided the time the program takes to run by more than a third, taking only 20558 milliseconds overall, using four threads. It also now used a full 100% of my computer's processor. However, because of how the Word Separator's algorithm works, it is not this simple. When testing more threads than processor cores/virtual cores, we still see improvement. If so, what is the true ideal thread count? I modified the Word Separator to take command line arguments and wrote a simple script to run it thousands of times with different thread counts while recording the program's run time. Here is the result¹ of testing up to 256 threads when analyzing Moby Dick:

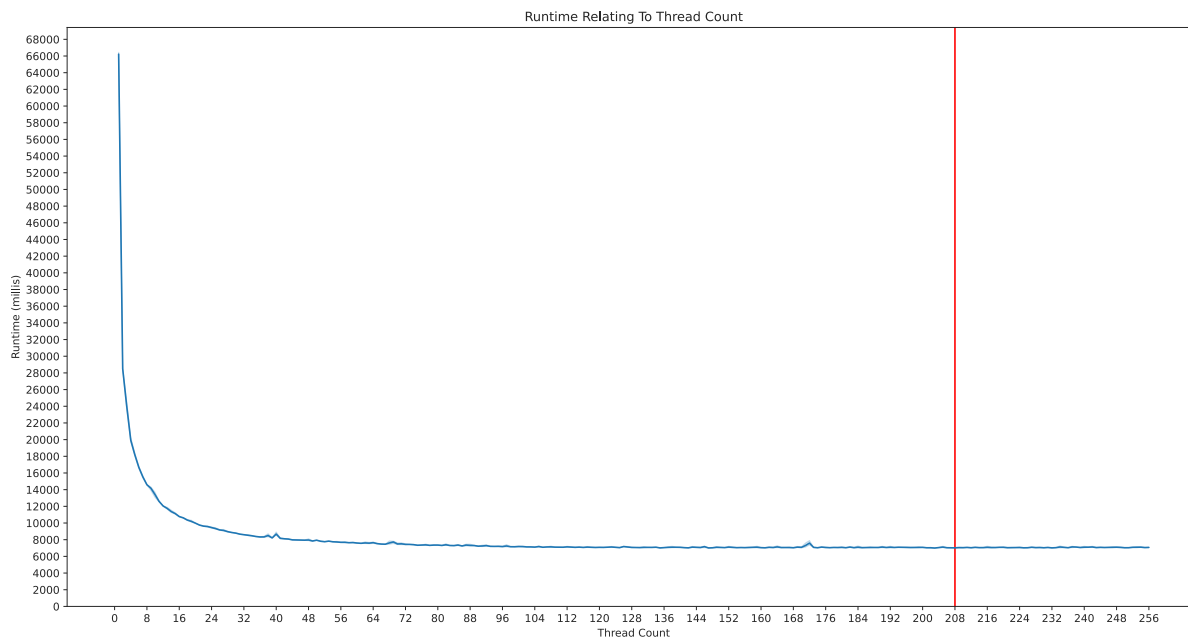


Figure 1: Result of testing the Word Separator with up to 256 threads.

As you can see², it achieves drastic improvements in run time within the first 64 threads, peaking around only 7000 milliseconds when using 208 threads.

¹The graphs in figures 1&2 display the total run time for the program for each thread count as the blue line. The red line indicates the fastest thread count.

²These figures were generated using the python library Matplotlib to create the graph and the library Seaborn library for nicer styling.

After this point, the performance cost of multi-threading starts to surpass the benefits of dividing the data into smaller chunks. This is much better performance than when I was previously only using four threads.

5.4 Why So Much Improvement?

This may seem a little counter-intuitive, though. Why, if the processor only has four cores, does the program benefit from running a lot more threads? If you refer back to Section 2.2 where I describe how the program works, you will see that each thread has to compare each word in its data-set to all the other words in the data-set to ensure there are no redundancies. This means that the number of calculations it has to compute increases non-linearly as the size of the data-set increases. This is a slightly algorithm from the original non-threaded program, which benefits not just from using all cores but also from dividing up the data-set for each thread into smaller chunks.

However, using multi-threading with other programs such as the word generator may provide different results. The word generator simply repeats the same function thousands of times, without having to do and redundancy checking.

5.5 Multi-Threading the Word Generator

After modifying the Random Word Generator V3 to take command line arguments and running a test similar to the test on the Word Separator, here are the results for generating 100000 random words from Moby Dick with up to 64 threads:

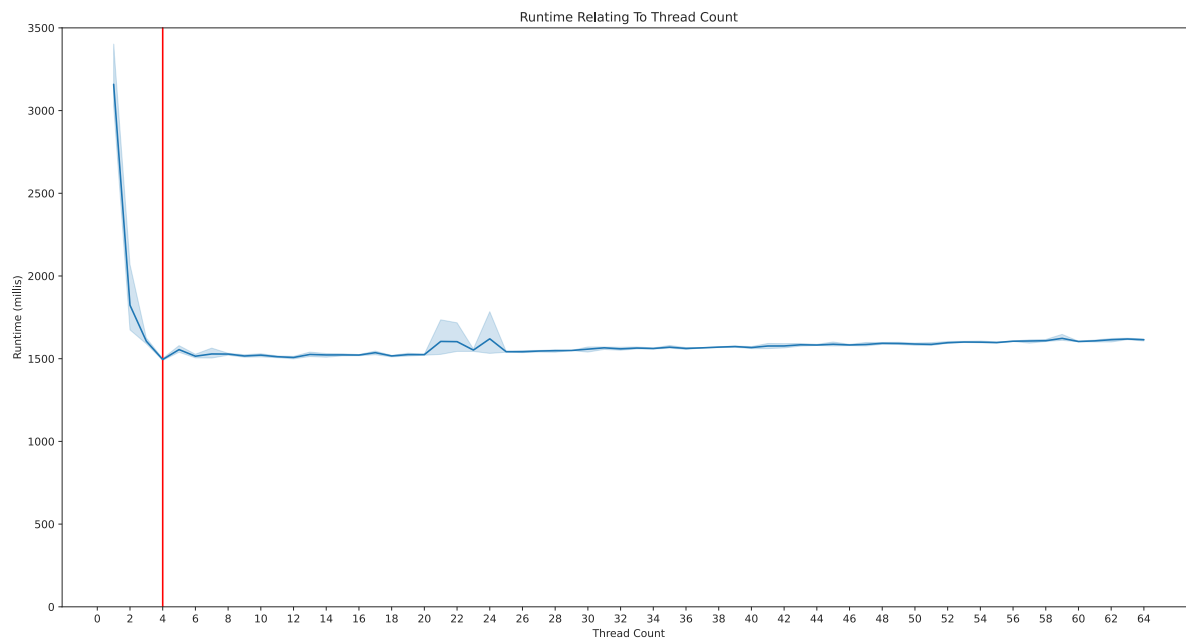


Figure 2: Result of testing the Word Generator with up to 64 threads.

This is as expected, with the fastest run time occurring when using 4 threads and small detrimental effects of using more threads.

6 Reflections

I had stopped using Python for a couple years, but after taking the time to write ten separate programs and a separate Python file with repeated functions I have learned a lot. Not only have I gotten much better at writing clean and efficient code, I have also learned how to make good use of libraries such as NumPy, Pandas, Matplotlib, and Seaborn.

Although I have made a lot of progress with my knowledge of Python, there remain things I could have done to make my code more "Pythonic". For my more complex programs, such as the Word Generator V3, it would have made sense to use classes to organize my code better and make my code more object oriented. Another thing I could have done to make my code more concise and possibly run more efficiently would have been to make better use of the libraries at hand. For example, instead of writing my own possibly inefficient function to collapse the list of words in the Word Separator into a list of unique words, I could have converted it to a Pandas dataframe and done the same thing with much fewer lines of code.

I also learned a little about processing and storing large amounts of data. For example, I created simple functions to store two dimensional lists in a text file similar to how a CSV file parser works. I used this, for example, to store the list of unique words and the quantities of each word. However, It would have made much more sense to use an actual CSV file and CSV parser. For processing data, I learned how to use NumPy arrays and Pandas dataframes to store, access, and manipulate large amounts of data. I also implemented multi-threaded processing to make the data processing run as efficiently as possible.

Finally, I have also used this opportunity to explore natural language processing and what makes a word sound like English. My progression through the different iterations of the word generators was an exercise in trying to figure out how to make words that are made up, but sound like they could actually exist in the English language. Through this, I figured out that there are certain combinations of letters that appear a lot in English words. By having the made up words borrow these combinations with a proportional frequency to the frequencies with which they appear in real English words, the made up words started to sound like actual English words.

Throughout this project, I have not only relearned the basics of Python, I have also learned a bit about data analysis as well as natural language processing. This has been a very interesting and educational project, and I will move on to apply much of what I have learned to image processing in another project.